

实验四 文件系统

1、实验目的

- 1) 理解文件系统的基本原理与实现机制。
- 2) 掌握基于 FUSE 的用户级文件系统开发。
- 3) 熟悉 inode、目录结构、位图等核心数据结构。
- 4) 掌握文件系统基本操作（创建、读写、删除等）的实现。
- 5) 理解文件系统中元数据管理、路径解析与大文件支持机制。

2、任务描述

本次实验基于 FUSE (Filesystem in Userspace) 库，实现一个用户级文件系统 Tiny File System (TFS)。文件系统所有数据均存储在一个 flat file（模拟真实存储设备）中，并通过 FUSE 将用户态文件系统挂载到指定目录，为用户提供标准文件操作接口。

本实验提供代码框架，主要包括：

- block.c: 块设备读写操作；
- block.h: 块大小等底层配置；
- tfs.c: 文件系统核心逻辑与 FUSE 接口实现；
- tfs.h: 超级块、inode、目录项、位图等数据结构定义；
- Makefile: 项目编译配置。

你需要在给定框架基础上补全 Tiny File System 的核心功能，并通过 benchmark 文件夹中编译后的测试程序（bitmap_check、simple_test、test_case）完成正确性验证。

3、实验内容

3.1 位图管理

文件系统需要维护 inode 位图和数据块位图，用于标记资源分配状态。需实现 inode 和数据块的位图管理，包括：

```
int get_avail_ino()
```

功能说明：遍历 inode 位图，找到空闲 inode，置位后返回 inode 编号。

```
int get_avail_blkno()
```

功能说明：遍历数据块位图，找到空闲数据块，置位后返回块号。

3.2 Inode 操作

inode 用于描述文件或目录元数据。请实现：

```
int readi(uint16_t ino, struct inode *inode)
```

功能说明：根据 inode 编号，从磁盘 inode 区读取 inode 信息到内存。

```
int writei(uint16_t ino, struct inode *inode)
```

功能说明：将内存中的 inode 信息写回磁盘 inode 区。

3.3 目录与路径解析

Tiny File System 采用树状目录结构。请实现：


```
struct fuse_file_info *fi)
```

功能说明：读取目录时调用，例如 `ls` 命令。接受文件或目录的路径作为输入。实现此函数需要读取 `inode` 并检查路径是否有效，然后将当前目录的所有目录项读取到输入缓冲区中。

```
static int tfs_mkdir(const char *path, mode_t mode)
```

功能说明：创建目录时（使用 `mkdir` 命令）调用。它接受目录的路径和模式作为输入。此函数首先需要将目录名和路径的基本名称分开。（例如，对于路径 `“/foo/bar/tmp”`，目录名为 `“/foo/bar”`，基本名称为 `“tmp”`）。然后应该读取目录名的 `inode` 并遍历其目录条目，查看是否已存在名称为基本名称的目录条目，如果存在，则返回负值；否则，必须将基本名称添加为目录。下一步是将新的目录条目添加到当前目录，分配一个 `inode`，并更新位图。

```
static int tfs_rmdir(const char *path)
```

功能说明：删除目录（`rmdir` 命令）时调用。它以路径目录作为输入。与 `mkdir` 类似，此函数首先需要分离路径的目录名和基本名称。（例如，对于路径 `“/foo/bar/tmp”`，目录名为 `“/foo/bar”`，基本名称为 `“tmp”`）。然后，它读取目录名的 `inode`，并遍历目录项，查看是否存在名称为基本名称的目录项。如果存在，则从当前目录中删除该目录，回收其 `inode`、数据块，并更新位图（更多详细提示可在框架代码中找到）。如果要删除的目录不存在，则返回负值。

```
static int tfs_create(const char *path, mode_t mode,  
struct fuse_file_info *fi)
```

功能说明：创建文件时调用，例如 `touch` 命令。它接受文件的路径和模式作为输入。此函数应首先分离路径的目录名和基本名称。（例如，对于路径 `“/foo/bar/a.txt”`，目录名为 `“/foo/bar”`，基本名称为 `“a.txt”`）。然后，它应该读取目录名的 `inode`，并遍历其目录项，查看是否已存在名称为 `base name` 的目录项。如果存在，则应返回负值。否则，`base name` 是有效的文件名，可以添加。下一步是将新目录项添加到当前目录，分配一个 `inode`，并更新位图（更详细的步骤可以在框架代码中找到）。

```
static int tfs_open(const char *path, struct fuse_file_info *fi)
```

功能说明：访问文件时调用。它以文件路径作为输入。它应该读取 `inode`，如果此路径有效，则返回 0，否则返回 -1。

```
static int tfs_read(const char *path, char *buffer, size_t size,  
off_t offset, struct fuse_file_info *fi)
```

功能说明：读取调用处理程序。它以文件路径、读取大小和偏移量作为输入。要实现此函数，首先，从输入的路径中读取该文件的 `inode`，然后使用该 `inode` 获取 `inode` 和数据块。接下来，将偏移量指定大小字节的数据块复制到内存区域指定位置。

```
static int tfs_write(const char *path, const char *buffer, size_t size,  
off_t offset, struct fuse_file_info *fi)
```

功能说明：写入调用处理程序。它接受文件路径、写入大小和偏移量作为输入。要执行写入操作，首先使用文件路径和 inode 读取 inode，定位数据块，然后将从偏移量到缓冲区指向的内存区域，写入 size 字节的数据。

```
static int tfs_unlink(const char *path)
```

功能说明：此函数在删除文件（rm 命令）时调用。它接受路径目录作为输入。首先，将路径的目录名和基本名称分开。（例如，对于路径“/foo/bar/a.txt”，目录名是“/foo/bar”，基本名称是“a.txt”）。接下来，读取目录名的 inode，并遍历其目录项，查看是否存在名称与基本名称相同的目录项。如果存在，则从当前目录中删除该目录，并回收其 inode、数据块和更新位图（更多详细提示可在框架代码中找到）。如果要删除的文件不存在，则返回负值。

3.5 大文件支持

为支持超过直接块指针容量的大文件，请实现 tfs.h 中 struct inode 中的单级间接索引 indirect_ptr。

要求：

- 文件支持一级间接块指针；
- 目录无需支持间接块。

4、建议实验步骤

- 1) 熟悉代码框架，理解超级块、inode、位图、目录项布局。
- 2) 实现 Block I/O 层 bio_read() / bio_write()。
- 3) 实现 inode 位图和数据块位图分配逻辑。
- 4) 实现 inode 读写函数 readi() / writei()。
- 5) 实现目录项管理与路径解析。
- 6) 实现 FUSE 文件系统接口并完成挂载测试。
- 7) 使用 benchmark 测试功能正确性。
- 8) 选做大文件支持并进行扩展测试。

5、实验测试

实验环境需要 FUSE library：

```
sudo apt install libfuse-dev fuse
```

构建和运行 Tiny File System 的步骤：

5.1 构建测试基础设施

```
> mkdir /tmp/<你的 ID>/  
> mkdir /tmp/<你的 ID>/mountdir
```

5.2 编译并运行 Tiny 文件系统

```
> cd code  
> make  
> ./tfs -s /tmp/<你的 ID>/mountdir
```

5.3 检查 Tiny 文件系统是否成功挂载

```
> findmnt (如果挂载成功, 可以看到以下信息)
/tmp/<你的 ID>/mountdir tfs fuse.tfs rw,nosuid,nodev,relatime,...
```

5.4 退出并卸载 (unmount) 文件系统

```
> fusermount -u /tmp/<你的 ID >/mountdir
```

5.5 Benchmark 测试

简单的基准测试 (Benchmark 文件夹中) 来测试 Tiny File System 的实现。

5.6 FUSE 调试技巧

1. 在 FUSE 库中进行调试并非易事, 因为我们不能总简单地使用 GDB 来调试 Tiny File System。但是, fuse 提供了 -d 选项; 当使用 -d 选项运行 Tiny File System 时, 它会在终端窗口中打印所有调试信息和跟踪信息。因此, 最佳方法是添加 print 语句, 并结合 GDB 来调试函数。
2. fuse 可能会返回一些错误
例如, Input/Output error or Transport endpoint is not connected), 因为某些 FUSE 文件处理程序在框架代码中尚未完全实现;
例如, 如果没有实现 tfs_getattr() 函数, 那么使用 cd 命令进入 Tiny File System 挂载点将会报错。

6、实验报告撰写要求

实验报告内容与形式:

- 1) 阐述 Tiny File System 的整体设计, 包括: 磁盘布局设计 (超级块、inode 区、位图区、数据块区)、inode 和目录项组织方式、路径解析实现流程。
- 2) 详细说明核心函数的实现思路。
- 3) 展示 benchmark 测试结果, 并分析测试过程中遇到的问题与解决方法。
- 4) 在实验报告最后以附录的形式分别粘贴 tfs.c 和 block.c 代码。**助教会对所有的实验报告进行查重, 请各组独立完成编码与实验报告。我们对抄袭零容忍。**

7、实验报告与代码提交要求 (**会影响最后评分, 请务必按格式要求提交**)

提交内容, 只提交三个文件:

- 1) tfs.c 文件
- 2) 实验报告 word 文档, 命名方式: OSLab4-成员 1 学号姓名-成员 2 学号姓名.docx
例如: OSLab4-B22035678 张三- B22035679 李四.docx
将以上三个文件打成一个压缩包, 命名方式: OSLab4-成员 1 学号姓名-成员 2 学号姓名.zip。
由班长或学委收集, 统一发给任课老师。

7、实验评分标准

给分点	分数
提交的代码能够正确编译	10%
所有给出的测试程序运行结果正确	30%

文件系统核心函数实现正确	20%
FUSE 接口功能实现正确	20%
代码注释详细、正确，实验报告文档内容详实	10%
实现支持超过直接块指针容量的大文件间接索引	10%
总计	100%

8、参考资料

1) Filesystem in Userspace:

https://en.wikipedia.org/wiki/Filesystem_in_Userspace

2) libfuse-github:

<https://github.com/libfuse/libfuse>

3) List of FSL FUSE Git repositories:

<https://www.filesystems.org/fuse/>